基于二叉堆与 Fibonacci 堆的 Dijkstra 最短路径算法比较

作者：陈昱年，宋诗雨，王宇轩
第四组。
陈昊老师。
代码与本轮实验完成日期：2026-10-16。

Chapter 1 问题描述

给定带非负弧权的有向道路网络和起终点，求一条总权值最小的路径，并比较优先队列结构对 Dijkstra 实际运行时间的影响。我们实现带位置映射的二叉堆和支持级联切割的 Fibonacci 堆；两者共享图、松弛过程、平局规则和查询序列，以尽量使测得差异来自优先队列。

输入采用 DIMACS shortest-path `.gr` 格式：`p sp n m` 声明规模，`a u v w` 描述有向弧，`c` 开头为注释。编号为 1..n，允许重边、自环和零权边。距离使用 64 位无符号整数；负权不满足 Dijkstra 前提而被拒绝。为保证有限最短路径不与不可达标记冲突，要求每条弧权不大于 `(UINT64_MAX-1)/max(n-1,1)`。

真实评测使用 USA-road-d 的 NY、BAY 和 COL 三个数据集，每个至少 1000 对查询。道路文件只列下载链接，不随报告上传。具体链接见 项目 README。

Chapter 2 数据结构与算法

2.1 CSR 图与统一状态

`Graph.offset` 有 n+1 个元素；顶点 u 的出弧保存在 `target`、`weight` 的 `[offset[u], offset[u+1])` 区间。读取时先保存弧并计数，再前缀求和、按起点填入 CSR，最后释放暂存弧。没有把有向边自动转成无向边，也没有改变输入权重。

每次搜索维护距离 `distance`、前驱 `previous`、定型标志 `settled` 和操作计数。顶点编号同时作为堆句柄，避免线性查找。两种堆按距离优先、顶点编号次之排序，使相同图和查询下的出堆顺序一致。首次发现一个顶点才插入，随后只 decrease-key；堆中不会出现过期的重复条目。

2.2 Dijkstra 伪代码

```
输入：CSR 图 G、源点 s、可选目标 t、优先队列类型 H
将全部 distance 设为 INF，previous 设为 -1，settled 设为 false
清空 H，distance[s] = 0，插入 s
当 H 非空：
    u = H.extract_min()
    settled[u] = true
    若指定目标且 u == t：停止
    对每条出弧 (u,v,w)：
        若 settled[v]：继续
        若 distance[u]+w 超出可表示的有限范围：继续
        candidate = distance[u]+w
        若 candidate < distance[v]：
            判断 v 是否首次被发现
```

```
            distance[v] = candidate，previous[v] = u
            首次发现则插入 v，否则执行 decrease_key(v)
返回目标距离及 previous 构造的路径，或完整单源距离
```

非负权保证最小候选出堆时距离定型。不能在首次遇到目标弧时停止；只有目标出堆后才能提前结束。未到达的节点保留 INF，源点到自身为 0。目标提前停止后，其他尚未定型节点的距离可能仍是暂定值。

2.3 二叉堆伪代码

`items` 保存完全二叉树的层序数组，`position[v]` 提供顶点 v 的当前位置。元素移动时同步更新位置映射。

```
insert_or_decrease(v):
    若 v 不在堆内：放到末尾，否则取 position[v]
    当 v 小于其父节点：将父节点下移并更新 position
    把 v 放到最终位置，更新 position[v]

extract_min():
    保存根，标记它不在堆内
    取末尾元素 x 填补根的空位
    当存在小于 x 的孩子：选择较小孩子上移，并更新 position
    将 x 放到最终位置，返回旧根
```

2.4 Fibonacci 堆伪代码

每个节点含父亲、一个孩子、左右兄弟、孩子数 degree、丢孩子标记 mark 和 active 状态。根链表及每组兄弟均为双向循环链表。实现用数组下标表达链接，顶点编号是稳定句柄，不为每次操作单独分配节点。

```
insert(v)：初始化单节点环，把 v 接入根链表，必要时更新 minimum

decrease_key(v):
    若 v 有父亲 p 且键值小于 p：
        从 p 的孩子环切下 v，p.degree--，将 v 加入根链表
        沿 p 向上：
            若 p 已是根：停止
            若 p 未标记：标记 p 并停止
            否则切下 p 并加入根链表，再处理原父亲
    必要时更新 minimum

extract_min():
    z = minimum
    将 z 的所有孩子提升为根，清除它们的 parent 和 mark
    从根链表删除 z
    逐个摘取根 x：
        当度数表中已有相同度数的根 y：
            将键较大的根挂到较小根下方，度数加一
        将合并结果放入对应度数槽
    用度数表重建根链表，并找出新的 minimum
    返回 z
```

遍历合并时先摘取根，再操作独立的度数表，避免原链表变化导致遗漏或重复遍历。由于节点数由 int 容量约束，度数表采用 `2*sizeof(int)*CHAR_BIT+1` 个槽，足以覆盖 Fibonacci 堆最大度数；断言检查内部不变量。Dijkstra 不需要对外的 meld/delete 任意节点接口，因此只实现实际使用的插入、减小键、删除最小值与重置。

2.5 性能实验方法

```
读取图，分别分配二叉堆和 Fibonacci 堆的搜索工作区
两种堆各预热一次
初始化相同非零种子的 xorshift64* 状态
重复 Q >= 1000 次：
    用拒绝采样生成 s,t，要求 s != t，允许查询对重复
    按查询奇偶交替两种堆的执行顺序
    分别计时执行相同的 (s,t) 查询
    在计时区间外逐对比较两个目标距离，若不一致立即失败
汇总时间、可达数、距离摘要及操作次数，写入 CSV
```

计时包含每次 O(n) 状态重置和堆重置，不包含图加载、初次分配、预热、随机数生成与输出。工作区跨查询复用；两种堆工作区同时存在，因而此实验不是峰值内存比较。用 `clock()` 计时，本机 `CLOCKS_PER_SEC=1000`，不同平台的时钟语义和分辨率需分别记录。

Chapter 3 测试与结果

3.1 环境与正确性

Windows x64，AMD Ryzen 9 8940HX with Radeon Graphics，MinGW-w64 GCC 15.2.0。所有三个 C 入口使用 `-std=c99 -O2 -Wall -Wextra -Wpedantic -Werror` 构建通过，最终构建无告警。测试不是只比较两种堆：小图另用独立 Floyd–Warshall 给出真值，堆另用线性扫描和结构审计验证。

用例组	目的与数量	结果
堆随机操作	30000 次插入、减小键、删除最小值，逐次和扫描真值比较	通过
堆结构审计	每步检查双向链接、父指针、度数、mark、active、堆序、位置映射	通过
显式级联切割	同一非根节点丢失两个孩子，分别检查标记与切割	通过
堆重置与复用	非空状态重置、空堆弹出、反复查询	通过
非法 DIMACS 输入	15 个缺头、弧数不符、非法端点、负权、溢出等输入	均被拒绝
特殊边和不连通	零权、自环、重边、孤立点、不可达	通过
64 位边界	两条大权弧之和为 UINT64_MAX-1	两种堆均精确返回
Floyd 独立对拍	100 张 1..32 顶点图，137400 次距离与路径检查	通过
终止条件	完整单源与目标出堆提前停止，全部起终点对照真值	通过
CLI	一条已知距离 8 的路径、同点、不可达、非法堆名和编号	通过

用例组	目的与数量	结果
基准输入约束	999 对查询、零种子	均拒绝
基准复现	合成图两次相同种子各 1000 对，比较除计时外全部 CSV 字段	完全一致

上述 137400 次包含两种堆、完整单源和逐对早停，不表示 137400 张图。合成图中包含不可达查询；真实路网本轮恰好 1000 对全部可达。测试压缩包只保留合成输入、验证日志与评测 CSV（脚本与代码见仓库 `code/`），不上传道路原始数据。

3.2 真实道路网络结果

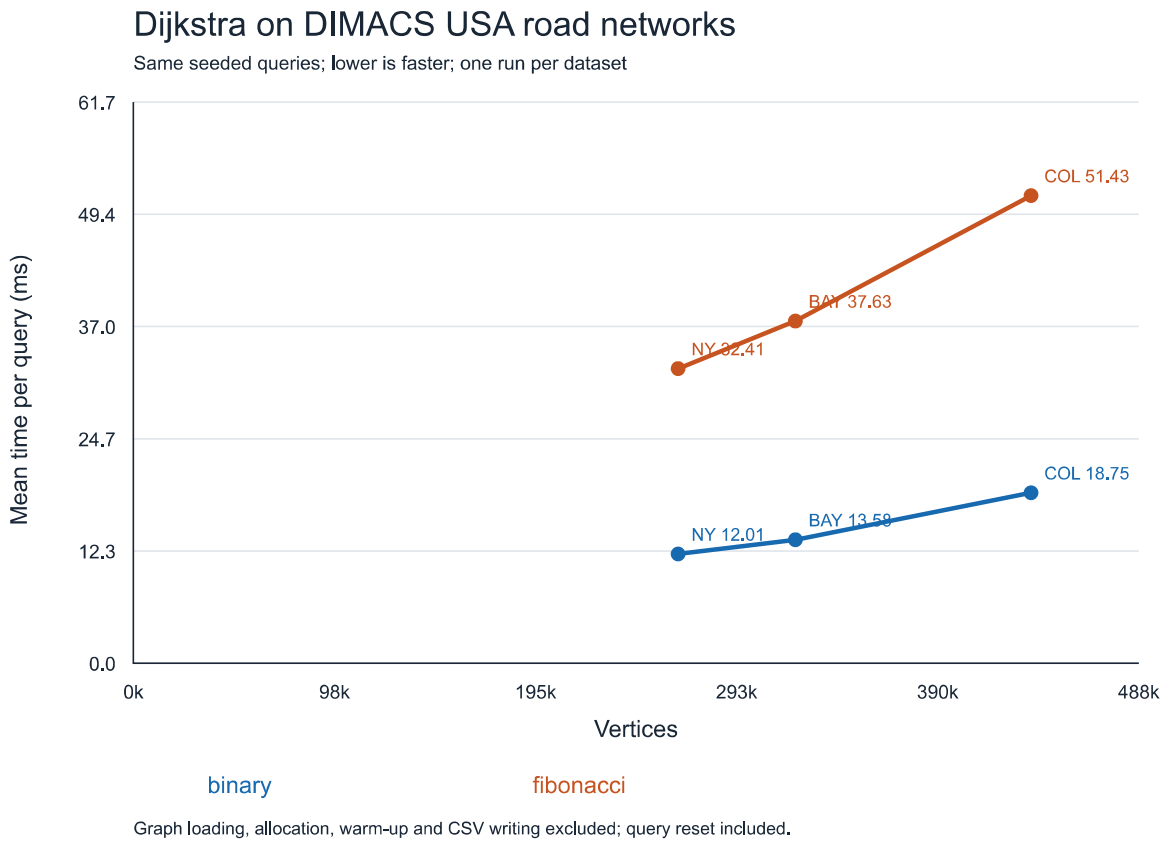
每份数据查询数均为 1000，种子均为 20261013；两种堆逐对距离一致。每个数据集各执行一轮。

路网	顶点 n	弧 m	二叉堆总秒数	Fibonacci 总秒数	二叉堆平均毫秒	Fibonacci 平均毫秒
NY	264346	733846	12.010	32.405	12.010	32.405
BAY	321270	800172	13.579	37.635	13.579	37.635
COL	435666	1057066	18.747	51.433	18.747	51.433

查询数恰为 1000，所以总秒数和平均毫秒数的数值相同，并非重复填表错误。

路网	可达查询数	距离摘要	总定型顶点数	总扫描弧数	decrease-key 次数
NY	1000	9337682968638045508	131698270	374443403	18796773
BAY	1000	12973616617807418106	162767154	408608921	13347723
COL	1000	14347840402608919281	220470963	540031571	16328701

表中摘要和操作次数在两种堆之间也完全一致。摘要为模 2^64 累积值，仅用于复现核对；正确性判断依据逐对距离比较及独立小图真值，而非摘要相同。



曲线以顶点数为横轴、每查询平均毫秒为纵轴。点之间的线只帮助阅读；不同路网的拓扑和弧顶点比不同，不能把它们当作仅改变 n 的受控渐近实验。

Chapter 4 复杂度与讨论

4.1 推导

图读取和 CSR 构建对 n 个计数槽、m 条弧各做常数次扫描，时间 O(n+m)，常驻空间 O(n+m)，读取过程还保留 O(m) 临时弧。每次搜索的距离、前驱、定型数组和堆重置需要 O(n) 时间。

设本次实际定型顶点为 r，扫描弧数为 e，发现的不同顶点为 d（d≤n）。每个顶点最多插入一次，出堆至多 r 次，成功的 decrease-key 不超过 e 次。二叉堆高度 O(log n)，每次插入、减小键或出堆最多沿树高移动，因此包含初始化的总时间为 O(n+(d+r+e)log n)，最坏写为 O((n+m)log n)。堆只保留一个顶点条目，空间 O(n)。

Fibonacci 堆采用势函数“根数 + 2×被标记节点数”：插入只增加一个根；级联切割虽然实际可沿多层执行，但消除标记降低势能，支付后续切割的成本，decrease-key 摊还 O(1)。按度合并后每个度数最多一个根，Fibonacci 子树规模下界保证最大度数 O(log n)，extract-min 摊还 O(log n)。因此总时间 O(n+d+e+r log n)，最坏 O(m+n log n)，空间同为 O(n)。这些是摊还界，不表示每次删除都只做 log n 次操作。

提前停止可能减少 r 和 e，但当前实现每次仍做 O(n) 重置；不能将早停查询的复杂度错误地写成只与到达节点有关。源点等于目标时搜索本身为常数工作，但包含重置仍为 O(n)。

完整基准另有一次 O(n+m) 图加载和两次预热；Q 次查询的耗时为各次搜索开销之和。测试中的 Floyd–Warshall 是 $O(n^3)$ 时间、$O(n^2)$ 空间，仅用于小图独立校验，未参与道路网络运行时间。

4.2 结果讨论

本轮三个稀疏路网中，Fibonacci 堆总时间约为二叉堆的 2.70、2.77、2.74 倍。虽然它的 decrease-key 摊还复杂度更好，但节点字段更多，循环链表维护、切割和根合并带来常数成本；二叉堆的数组访问更紧凑。相同探

索顺序下操作计数一致，表明这轮差异不是源于某个算法搜索了更多节点。

“常数与缓存行为造成差异”是结合实现的解释，不是硬件性能计数器实测结论。我们未采集 cache miss，也未实测峰值内存，不能把这些推测写成实测指标。路网随规模增长耗时上升，但只含三个不同拓扑的样本，不能据此验证渐近复杂度或声称某堆永远最优。

4.3 局限与复现

- 每份数据只测一轮；没有多次独立重复、置信区间和 CPU 绑核控制。更严谨的性能研究应重复多轮并报告中位数、离散度。
- `clock()` 分辨率在本机为 1 ms；它在不同 C 运行库中可能表示 CPU 时间或经过时间，跨平台数据不应混合。小图计时可能为零，因此不用于速度结论。
- 使用原始单向 Dijkstra，没有 A*、双向搜索或道路预处理；结论仅适用于这里的实现。
- 未实测 Linux 构建、完整 USA 网络、旅行时间权重版本和额外堆类型；报告待评审，尚未生成定稿 PDF。

复现时按项目 README 下载解压数据，使用相同编译参数、查询数和种子运行 `benchmark`；CSV 中距离摘要和操作计数应一致，时间可能变化。自动回归入口为 `code/tests/run_tests.py`，生成曲线的入口为 `documentation/assets/plot_results.py`（绘图脚本属于文档工具，因此与报告放在同一目录下）。原始 CSV、测试日志、数据哈希和合成输入收录于测试结果压缩包。